

Sales Analyzer — Database Reference

SQLite 3 · Python / Flask · May 16, 2026

1. Overview

Sales Analyzer is a Flask-based web platform where sales representatives log transactions and view analytics dashboards. Administrators see consolidated metrics across all users. Data is stored in a two-table SQLite database (**users** + **sales**) accessed via raw SQL using Python's built-in *sqlite3* module — no ORM.

2. Schema & Relationships

One user owns many sales records (1 : N). The **sales.user_id** foreign key references **users.id**. No CASCADE DELETE is defined — deleting a user leaves their sales records intact.

Relationship: `users.id` → `sales.user_id` (One-to-Many)

3. Table: users

Central identity and authentication store. Every registered person — rep or admin — has exactly one row.

Field	Type	Constraint	Description
id	INTEGER	PK, AUTOINCREMENT	Unique auto-incremented user identifier
name	TEXT	NOT NULL	Full display name
email	TEXT	UNIQUE, NOT NULL	Login credential
password	TEXT	NOT NULL	Werkzeug PBKDF2-SHA256 hash — never plain-text
role	TEXT	DEFAULT 'user'	'user' (sales rep) or 'admin' (administrator)
allow_admin_view	INTEGER	DEFAULT 0	Boolean: grants non-admin access to admin dashboards
monthly_goal	REAL	DEFAULT 50000	Personal monthly revenue target for progress tracking
created_at	TEXT	DEFAULT datetime('now')	Account creation timestamp (ISO-8601)

4. Table: sales

Core transactional table. Each row represents one sale. **total** and **profit** are pre-computed and stored for fast aggregation.

Field	Type	Constraint	Description
id	INTEGER	PK, AUTOINCREMENT	Unique transaction identifier
user_id	INTEGER	FK → users(id)	Owning salesperson
product	TEXT	NOT NULL	Product name or description (free-text)
category	TEXT	NOT NULL	Product category label (free-text, no lookup table)
date	TEXT	NOT NULL	Sale date stored as YYYY-MM-DD string
rate	REAL	NOT NULL	Selling price per unit
cost_price	REAL	DEFAULT 0	Cost/purchase price per unit
quantity	INTEGER	NOT NULL	Number of units sold
total	REAL	NOT NULL	rate × quantity (pre-computed, stored)
profit	REAL	DEFAULT 0	(rate – cost_price) × quantity (pre-computed, stored)

5. Key Data Flows

Flow	Detail
Registration	INSERT into users; password hashed with Werkzeug PBKDF2-SHA256; role → 'user'; monthly_goal → 50,000
Login	SELECT users WHERE email = ?; verify hash via check_password_hash(); store id + role in Flask session
Add Sale	App computes total & profit; INSERT into sales with session user_id as FK